Design of Algorithms

for

EcoMesh

Distributed Peer-to-Peer Disaster Relief Communication Mesh

Prepared by

Miti Shah	2024300230	miti.shah24@spit.ac.in
Twissha Shah	2024300231	twissha.shah24@spit.ac.in
Tanvi Sheth	2024300239	tanvi.sheth24@spit.ac.in

Instructor:	Prof. Anuj Tawari
Course:	Mini Project I
Date:	24-09-26

Table of Contents

1 Introduction **5**

1.1 Purpose . . . 5

1.2 Scope . . . 5

1.3 Source Document . . . 5

1.4 Notation and Conventions . . . 6

1.5 Parameters and Default Values . . . 6

1.6 Index of Algorithms . . . 7

1.7 Implementation Status . . . 7

2 Algorithm 1: Neighbour Discovery and Link Quality Estimation **8**

2.1 Purpose . . . 8

2.2 Inputs and Outputs . . . 8

2.3 Pseudocode . . . 9

2.4 Complexity . . . 9

2.5 Design Notes . . . 9

2.6 Requirements Satisfied . . . 10

3 Algorithm 2: Link-State Dissemination and Graph Construction **10**

3.1 Purpose . . . 10

3.2 Inputs and Outputs . . . 10

3.3 Pseudocode . . . 10

3.4 Complexity . . . 11

3.5 Design Notes . . . 11

3.6 Requirements Satisfied . . . 11

4 Algorithm 3: Least-Cost Route Computation **11**

4.1 Purpose . . . 11

4.2 The Cost Function . . . 12

4.3 Worked Example . . . 12

4.4 Inputs and Outputs . . . 12

4.5 Pseudocode . . . 12

4.6 Complexity . . . 13

4.7 Requirements Satisfied . . . 13

5 Algorithm 4: Packet Forwarding and Relay Decision **13**

5.1 Purpose . . . 13

5.2 Inputs and Outputs . . . 14

5.3 Pseudocode . . . 14

5.4 Complexity . . . 15

5.5 Design Notes . . . 15

5.6 Requirements Satisfied . . . 15

6 Algorithm 5: Priority Scheduling with Per-Hop Pre-emption **15**

6.1 Purpose . . . 15

6.2 Inputs and Outputs . . . 15

6.3 Pseudocode . . . 16

6.4 Complexity . . . 16

6.5 Requirements Satisfied . . . 16

7 Algorithm 6: Duplicate Suppression and Loop Prevention **16**

7.1 Purpose . . . 17

7.2 Inputs and Outputs . . . 17

7.3 Pseudocode . . . 17

7.4 Complexity . . . 17

7.5 Requirements Satisfied . . . 17

8 Algorithm 7: Acknowledgement and Bounded Retransmission **18**

8.1 Purpose . . . 18

8.2 Inputs and Outputs . . . 18

8.3 Pseudocode . . . 18

8.4 Complexity . . . 19

8.5 Design Notes . . . 19

8.6 Requirements Satisfied . . . 19

9 Algorithm 8: Store-and-Forward Retention and Re-attempt **19**

9.1 Purpose . . . 19

9.2 Inputs and Outputs . . . 19

9.3 Pseudocode . . . 20

9.4 Complexity . . . 20

9.5 Design Notes . . . 20

9.6 Requirements Satisfied 21
10 Algorithm 9: Message Protection and Verification 21
10.1 Purpose 21
10.2 Inputs and Outputs 21
10.3 Pseudocode 21
10.4 Complexity 22
10.5 Requirements Satisfied 22
11 Algorithm 10: Terminal Selection for Rescue-Addressed Traffic 22
11.1 Purpose 22
11.2 Inputs and Outputs 22
11.3 Pseudocode 22
11.4 Complexity 23
11.5 Requirements Satisfied 23
12 Complexity Summary 23
13 Traceability to the SRS 24

Revision History

Name	Date	Reason For Changes	Version
Group 20	24-09-26	Initial release of the algorithm design	1.0
Group 20	25-09-26	Corrected cost-function weights, TTL and timeout defaults to match the implemented code and the master project plan; added an implementation-status section distinguishing built-and-tested behaviour from designed-but-not-yet-coded behaviour; rebranded to EcoMesh	1.1
Group 20	11-10-26	Closed four of the six gaps recorded in v1.1's implementation-status section: link-state re-flooding, ACK-timeout failure feedback into the cost function, store-and-forward count/age bounds, and public-key distribution with Ed25519 signing end to end. Updated every affected status note accordingly; only link-quality measurement and X25519 key agreement remain unimplemented.	1.2

1 Introduction

This document specifies the algorithms that realise the functional requirements of the **EcoMesh** Distributed Peer-to-Peer Disaster Relief Communication Mesh. Each algorithm is presented with its purpose, its inputs and outputs, structured pseudocode, an analysis of its time and space behaviour, and a reference to the specific requirements of the Software Requirements Specification that it satisfies.

1.1 Purpose

The purpose of this document is to fix the logic of the system before implementation begins, so that the behaviour of every node is defined unambiguously and independently of the programming language used. It serves as the direct reference for coding, as the basis for deriving unit and integration test cases, and as the record against which the correctness and efficiency of the finished system can be assessed.

1.2 Scope

Ten algorithms are specified. Together they cover peer discovery, topology dissemination, route computation, packet forwarding, priority scheduling, duplicate suppression, reliable delivery, delay-tolerant retention, message security and terminal selection. Every functional requirement group of Section 4 of the Software Requirements Specification is realised by at least one of them; the mapping is given in Section 13.

1.3 Source Document

All algorithms are derived from the *Software Requirements Specification for EcoMesh*, Version 1.1, Group 20, Department of Computer Engineering, SPIT. Where a requirement fixes a behaviour, the pseudocode implements it exactly; where the specification leaves a value open, a working default is given in Section 1.5 and is marked as provisional.

1.4 Notation and Conventions

The pseudocode uses the following conventions:

- <- denotes assignment and = denotes comparison.
- `procedure` introduces a routine that returns no value; `function` introduces one that does.
- Text following // is a comment and is not executed.
- `NOW()` returns the current local clock reading in seconds. Nodes are not assumed to share a synchronised clock; every timing comparison in this document is made against the node's own clock only.
- `NIL` denotes the absence of a value and `INFINITY` denotes an unreachable cost.
- Module names in capitals — ROUTING, TRANSPORT, SECURITY, STORE, QUEUE, STOREFWD, UI — refer to the components defined in the UML Design Specification.
- Complexity is stated in terms of *V*, the number of nodes in the network graph, *E*, the number of links, and |*N*|, the number of direct neighbours of the node executing the algorithm.

1.5 Parameters and Default Values

The following parameters govern the behaviour of the algorithms. The values given are the working defaults **as actually configured in the current implementation** (see the corresponding modules in `mesh/core/`), except where marked provisional, for the specified deployment of up to fifty nodes.

Symbol	Parameter	Default	Used In
T_b	Discovery beacon interval	3 s	Algorithm 1
W	Sliding window for link quality	10 beacons (design target — see §1.7)	Algorithm 1
T_out	Silence timeout before a neighbour is declared lost	10 s	Algorithm 1
T_ls	Link-state advertisement interval	5 s	Algorithm 2
H	Base hop penalty in the link cost function	1.0	Algorithm 3
alpha	Weight applied to measured link quality	5.0	Algorithm 3
beta	Weight applied to recent delivery failures	3.0	Algorithm 3
TTL_0	Initial hop allowance carried by a new packet	10 hops	Algorithm 4
T_keep	Retention period for observed message identifiers	300 s	Algorithm 6
T_ack	Acknowledgement timeout	4 s	Algorithm 7
A_max	Maximum number of delivery attempts	3	Algorithm 7
C_max	Bound on the retention queue, by message count	500 messages	Algorithm 8
T_hold	Maximum retention period for an undeliverable message	1800 s	Algorithm 8

Both bounds are enforced by the current build (§1.7). Their exact numeric values are still recorded as item TBD-5 in Appendix C of the Software Requirements Specification and remain provisional, subject to confirmation by measurement during testing — a different kind of “provisional” than an unimplemented mechanism.

1.6 Index of Algorithms

No.	Algorithm	SRS Feature
1	Neighbour Discovery and Link Quality Estimation	4.1 Peer Discovery
2	Link-State Dissemination and Graph Construction	4.4 Adaptive Routing
3	Least-Cost Route Computation	4.4 Adaptive Routing
4	Packet Forwarding and Relay Decision	4.3 Multi-Hop Message Forwarding
5	Priority Scheduling with Per-Hop Pre-emption	4.5 Priority-Based Message Handling
6	Duplicate Suppression and Loop Prevention	4.6 Reliability Mechanisms
7	Acknowledgement and Bounded Retransmission	4.6 Reliability Mechanisms
8	Store-and-Forward Retention and Re-attempt	4.7 Store-and-Forward Delivery
9	Message Protection and Verification	4.8 Message Security
10	Terminal Selection for Rescue-Addressed Traffic	4.4 Adaptive Routing

1.7 Implementation Status

This document specifies the target design for every algorithm, consistent with the Software Requirements Specification. As of the current build, nearly all of it is implemented and covered by automated tests; two narrowly-scoped gaps remain, listed below. Both are legitimate content for a design document — the SRS defines what the finished system must do — but the distinction matters for anyone extending the code or answering questions about it, so it is stated here explicitly rather than left to be discovered.

Implemented and tested:

- Algorithm 1 — beacon-based discovery, the neighbour table, silence-based failure detection, and public-key distribution via HELLO (excluding the sliding-window link-quality measurement, below).
- Algorithm 2 — link-state exchange *with* re-flooding: a report is merged locally and passed on to a node's other neighbours, with origin + sequence numbers rejecting stale replays, so the graph is genuinely network-wide rather than stopping two hops out. Proven with a test where a node originates a message three hops away purely through `Node.send()`, with no manually addressed first hop.
- Algorithm 3 — Dijkstra over the weighted graph, with the cost function using the corrected coefficients in §1.5.
- Algorithm 4 — all four guards (structure, duplicate, loop, hop allowance) and the deliver/relay branches, including that a relay never decrypts a payload it forwards.
- Algorithm 5 — the priority heap, re-applied at every hop.
- Algorithm 6 — duplicate suppression with a time-bounded record.

- Algorithm 7 — send, await-acknowledgement, timeout-triggered retry with route recompute, bounded attempts, and failure feedback into the cost function (`ROUTING.record_failure` now actually called on every timeout, taking effect immediately rather than waiting for the next gossip round).
- Algorithm 8 — retention and re-attempt, now with both bounds enforced: count-based eviction removes the lowest-priority message first (SNF-5), and a separate age-based EXPIRE sweep removes anything held past T_hold regardless of priority.
- Algorithm 9 — both halves. AES-256-GCM payload encryption end to end, opt-in via a pre-shared session key; and Ed25519 header signing/verification end to end, opt-in via a per-node keypair, with public keys distributed through HELLO and propagated network-wide via the Algorithm 2 re-flooding above (a node signed by someone more than one hop away still verifies correctly). One deliberate deviation from the pseudocode in §10.3: the signature covers the fields that stay constant across every hop, not the literal full header, since ttl/hop_count/path are mutated by every relay by design and a signature over them could never verify past the first hop. X25519 per-message key agreement is not implemented; encryption uses a single pre-shared session key (the Phase 1 scope §10 itself allows).
- Algorithm 10 — terminal selection over reachable GATEWAY/RESCUE nodes, folded directly into route computation.

Designed here, not yet implemented:

- **Link-quality measurement (Algorithm 1, DIS-5).** The sliding-window beacon-reception ratio is specified here; the current build has every link default to a perfect quality of 1.0 and exposes a `record_link_quality()` hook for this measurement to feed once implemented.
- **X25519 per-message key agreement (Algorithm 9).** See above — the implemented encryption uses a single pre-shared session key rather than a key derived per message pair.

2 Algorithm 1: Neighbour Discovery and Link Quality Estimation

2.1 Purpose

A node must learn which other nodes are within direct communication range without any manual entry of addresses, must measure how reliable each of those links is, and must notice promptly when a neighbour stops responding. The algorithm achieves all three from a single periodic beacon: arrival of a beacon proves reachability, the proportion of expected beacons actually received measures quality, and continued silence indicates failure.

2.2 Inputs and Outputs

Symbol	Direction	Meaning
T_b	input	Beacon transmission interval in seconds.
W	input	Length of the sliding observation window, in beacon periods.
T_out	input	Silence period after which a neighbour is declared unreachable.
N	output	Neighbour table: identifier, address, role, public key, last-seen time and measured link quality.

Symbol	Direction	Meaning
events	output	node_arrived and node_lost notifications raised to the routing engine and the interface.

2.3 Pseudocode

Algorithm 1 — DiscoverNeighbours

```
// ---------- transmitting task, executed once every T_b seconds ----------
procedure SEND_BEACON()
    b.node_id    <- self.node_id
    b.role       <- self.role              // USER | GATEWAY | RESCUE
    b.public_key <- self.public_key        // distributed with the beacon
    b.seq        <- self.beacon_seq
    self.beacon_seq <- self.beacon_seq + 1
    TRANSPORT.broadcast(ENCODE(b))         // UDP multicast, or seed list

// ---------- receiving task, executed on every arriving beacon ----------
procedure ON_BEACON(b, src_addr)
    if not VALID(b) then
        return                            // malformed, ignore silently
    if b.node_id not in N then
        N[b.node_id]       <- NEW Neighbour(b.node_id, src_addr, b.role,
                                            b.public_key)
        N[b.node_id].window <- empty queue of capacity W
        RAISE node_arrived(b.node_id)      // triggers store-and-forward
    n            <- N[b.node_id]
    n.address    <- src_addr               // address may have changed
    n.role       <- b.role
    n.last_seen  <- NOW()
    PUSH(n.window, b.seq)                  // oldest entry drops out
    n.quality    <- COUNT_DISTINCT(n.window) / W   // 0.0 .. 1.0

// ---------- failure detection task, executed once every T_b seconds ----------
procedure DETECT_FAILURE()
    for each n in N do
        if NOW() - n.last_seen > T_out then
            REMOVE n from N
            ROUTING.remove_edges(self.node_id, n.node_id)
            ROUTING.recompute()            // routes must not use a dead link
            RAISE node_lost(n.node_id)
```

2.4 Complexity

SEND_BEACON is O(1) in time and produces one datagram per period regardless of network size, because it is broadcast rather than addressed. ON_BEACON is O(1) amortised, the sliding window being a fixed-capacity queue. DETECT_FAILURE is O(|N|) per period. Space is O(|N| × W), which is bounded because both the neighbour count and the window length are bounded.

2.5 Design Notes

- Link quality is a measured quantity, not an assumption. A link that delivers seven of the last ten beacons scores 0.7, and that figure feeds directly into the cost function of Algorithm 3. *This measurement is designed here but not yet implemented — see §1.7; every link currently defaults to a quality of 1.0.*

- The public key travels in the beacon, so no key distribution step is needed before two nodes can exchange protected traffic (SEC-6). *Not yet implemented — see §1.7.*
- Where the local segment does not permit multicast, the same procedure runs against a configured seed list of peer addresses (DIS-7); no other part of the algorithm changes.

2.6 Requirements Satisfied

DIS-1, DIS-2, DIS-3, DIS-4, DIS-5, DIS-6, DIS-7, SEC-6, REL-8.

3 Algorithm 2: Link-State Dissemination and Graph Construction

3.1 Purpose

Discovery tells a node only about its immediate neighbours. To compute a route across several hops, a node needs a picture of the whole network. Each node therefore periodically advertises its own neighbour list, and these advertisements are propagated so that every node can assemble the same weighted graph. Sequence numbers make the propagation terminate and keep stale information from overwriting fresh information.

Implementation status: *the re-flooding step below and the sequence-number staleness check are both implemented (§1.7) — origin + sequence number are carried in the payload itself so a report can be safely re-flooded past the first hop and a stale replay rejected.*

3.2 Inputs and Outputs

Symbol	Direction	Meaning
T_ls	input	Interval between link-state advertisements.
N	input	Local neighbour table produced by Algorithm 1.
G	output	Network graph. G[u][v] holds the measured quality of the link from u to v.
SEQ	output	Highest advertisement sequence number seen from each originating node.

3.3 Pseudocode

Algorithm 2 — ExchangeLinkState

```
// ---------- advertising task, executed once every T_ls seconds ----------
procedure ADVERTISE()
    r.origin <- self.node_id
    r.seq    <- self.ls_seq
    self.ls_seq <- self.ls_seq + 1
    r.links  <- { (n.node_id, n.quality) : n in N }   // own adjacency
    for each n in N do
        TRANSPORT.send(n.address, ENCODE(r))          // neighbours only

// ---------- executed on every arriving link-state report ----------
procedure ON_LINK_STATE(r)
    if r.origin = self.node_id then
```

```
        return                              // our own report, came back
    if r.origin in SEQ and r.seq <= SEQ[r.origin] then
        return                              // stale or already seen
    SEQ[r.origin] <- r.seq
    old <- G[r.origin]
    G[r.origin] <- r.links                  // replace, never merge

    for each n in N do                      // controlled re-flooding
        if n.node_id != r.origin then
            TRANSPORT.send(n.address, ENCODE(r))

    if old != G[r.origin] then
        ROUTING.recompute()                 // topology actually changed

// ---------- executed when Algorithm 1 reports a lost neighbour ----------
procedure ON_NODE_LOST(id)
    REMOVE id from G                        // drop its outgoing edges
    for each u in G do
        REMOVE id from G[u]                 // drop its incoming edges
    ROUTING.recompute()
```

3.4 Complexity

ADVERTISE is O(|N|) in time and sends |N| datagrams per period. ON_LINK_STATE is O(|N|) for the re-flooding step plus O(deg(origin)) for the adjacency replacement. Because a report is re-flooded only when its sequence number is higher than any seen before, each report crosses each link at most once and propagation is guaranteed to terminate. Space is O(V + E). This matches the current build exactly (§1.7).

3.5 Design Notes

- The adjacency of an originator is *replaced* rather than merged. A link that disappears from a report is therefore removed from the graph, which is what allows a failed link to be forgotten.
- Routes are recomputed only when the merged report actually changes the graph. A periodic report that repeats what is already known costs nothing beyond its own propagation.
- Convergence after a change is bounded by the network diameter multiplied by T_ls, which is what SRS 5.1 expresses as a 15-second bound. This bound assumes re-flooding is in place.

3.6 Requirements Satisfied

RTE-1, RTE-2, RTE-5, RTE-8, REL-8.

4 Algorithm 3: Least-Cost Route Computation

4.1 Purpose

This is the central algorithm of the system. Given the network graph assembled by Algorithm 2, it computes the least-cost path from this node to every other node, and from each path extracts the first hop, which is all a forwarding node needs to know. The cost of a link is not fixed at one unit as it would be under a hop-count metric; it combines a base hop penalty, the measured quality of the link, and the number of recent delivery failures observed on it. A short path over unreliable links can therefore cost more than a longer path over good links, and the algorithm will choose the longer path.

4.2 The Cost Function

For a link from node *u* to node *v*, the cost is defined as:

$$\text{cost}(u, v) = H + \text{alpha} \times (1 - q(u, v)) + \text{beta} \times f(u, v)$$

where q(u, v) is the measured link quality in the range 0.0 to 1.0 produced by Algorithm 1, f(u, v) is the number of delivery failures recorded on that link within the recent window, H is the base hop penalty that makes a shorter path preferred when quality is equal, and alpha and beta are the weights given in Section 1.5. Every term is non-negative and H is strictly positive, so all costs are positive and Dijkstra's algorithm is applicable.

4.3 Worked Example

Using the implemented default values H = 1.0, alpha = 5.0 and beta = 3.0, consider two paths from node A to node D with no recorded failures:

Path	Hops	Link Qualities	Total Cost	Chosen
A → X → D	2	0.40, 0.45	4.00 + 3.75 = 7.75	No
A → B → C → D	3	0.95, 0.95, 0.95	1.25 × 3 = 3.75	Yes

A hop-count metric would have selected the two-hop path and delivered the message over two links that drop more than half of what they carry. The weighted metric selects the three-hop path, which is longer but far more likely to deliver. This is the behaviour required by RTE-4 and is the distinguishing property of the routing design, and is proven directly by an automated test in the current build (`test_weighted_routing_avoids_lossy_shortcut`).

4.4 Inputs and Outputs

Symbol	Direction	Meaning
G	input	Network graph with measured link qualities.
F	input	Recent delivery failure counts per link.
H, alpha, beta	input	Cost function parameters from Section 1.5.
dist[]	output	Least cost from this node to every reachable node.
next[]	output	First hop on the least-cost path to every reachable node, or NIL if unreachable.

4.5 Pseudocode

Algorithm 3 — ComputeRoutes (Dijkstra, single source)

```
function COST(u, v)
    q <- G[u][v]                          // measured quality, 0.0 .. 1.0
    return H + alpha * (1 - q) + beta * F[u][v]

procedure COMPUTE_ROUTES()
    for each v in G do
        dist[v] <- INFINITY
        next[v] <- NIL
```

```
    dist[self.node_id] <- 0
    Q <- min-priority queue holding the pair (0, self.node_id)
    S <- empty set                        // vertices already finalised

    while Q is not empty do
        (d, u) <- EXTRACT_MIN(Q)
        if u in S then
            continue                      // stale entry, already done
        S <- S + { u }

        for each v adjacent to u in G do
            if v in S then
                continue
            c <- d + COST(u, v)
            if c < dist[v] then
                dist[v] <- c
                // propagate the first hop instead of walking back later:
                if u = self.node_id then
                    next[v] <- v           // v is a direct neighbour
                else
                    next[v] <- next[u]     // inherit u's first hop
                INSERT(Q, (c, v))

function NEXT_HOP(dst)
    if dst not in next or next[dst] = NIL then
        return NIL                        // no route at this moment
    return next[dst]
```

4.6 Complexity

With a binary min-heap the running time is O((V + E) log V). Propagating the first hop during relaxation, rather than reconstructing each path afterwards from a predecessor array, keeps the whole computation within that bound and removes a separate O(V × d) traversal, where d is the path depth. Space is O(V + E) for the graph plus O(V) for the distance, next-hop and queue structures. For the specified fifty-node deployment this is a few milliseconds of work, which is why a full recomputation on every topology change is affordable. The implemented version reconstructs the path from predecessor pointers after the fact rather than propagating the first hop during relaxation; this is asymptotically equivalent and does not change the bound.

4.7 Requirements Satisfied

RTE-3, RTE-4, RTE-5, RTE-8.

5 Algorithm 4: Packet Forwarding and Relay Decision

5.1 Purpose

This algorithm runs at every node on every arriving packet. It decides whether the packet is to be discarded, delivered locally, or relayed onward, and it is the point at which every safety guarantee of the system is enforced: structural validation, duplicate suppression, loop detection and the hop allowance. The order of the checks is deliberate — the cheapest and most protective tests are applied first, and nothing is decrypted or forwarded until the packet has passed all of them.

5.2 Inputs and Outputs

Symbol	Direction	Meaning
p	input	Packet received from the transport layer.
SEEN	input	Cache of recently observed message identifiers (Algorithm 6).
action	output	The packet is delivered locally, enqueued for forwarding, retained, or discarded with a logged reason.

5.3 Pseudocode

Algorithm 4 — HandlePacket

```
procedure ON_PACKET(p)

    // ---------- guard 1: structure ----------
    if not PARSE_AND_VALIDATE(p) then         // magic, version, lengths
        LOG(discard, p, "malformed header")
        return

    // ---------- guard 2: duplicate ----------
    if p.msg_id in SEEN then
        LOG(discard, p, "duplicate")
        return

    // ---------- guard 3: loop ----------
    if self.node_id in p.path then
        LOG(discard, p, "loop detected")
        return

    // ---------- guard 4: hop allowance ----------
    if p.ttl = 0 then
        LOG(discard, p, "hop allowance exhausted")
        return

    SEEN[p.msg_id] <- NOW()                   // record before acting on it

    // ---------- delivery branch ----------
    if p.dst = self.node_id
       or (p.dst = RESCUE_SERVICE and self.role = RESCUE) then
        if not SECURITY.verify(p) then
            LOG(discard, p, "signature verification failed")
            return
        text <- SECURITY.decrypt(p.payload)   // fails closed on bad tag
        if text = NIL then
            LOG(discard, p, "authentication tag failed")
            return
        STORE.save_message(p, text, DELIVERED)
        UI.push(p, text)
        SEND_ACK(p)                           // back along p.path reversed
        return

    // ---------- relay branch ----------
    p.ttl  <- p.ttl - 1
    p.path <- p.path + [ self.node_id ]
    h <- ROUTING.next_hop(p.dst)
    if h = NIL then
        STOREFWD.retain(p)                    // no route at this moment
        return
```

```
    QUEUE.enqueue(p, p.priority)              // Algorithm 5 orders it
    LOG(forward, p, h)
```

5.4 Complexity

All four guards and both branches are O(1) apart from the loop check, which is O(|path|) and is bounded above by the initial hop allowance TTL_0. The work done per packet is therefore constant with respect to the size of the network, which is what allows a node to relay traffic without its own responsiveness degrading as the mesh grows.

5.5 Design Notes

- The relay branch never calls `SECURITY.decrypt`. A relaying node therefore cannot recover the plaintext of a message it forwards, which is requirement SEC-2 and FWD-7 expressed in code, and is directly proven by an automated test in the current build (`test_relay_never_sees_plaintext_payload`).
- Three independent mechanisms prevent a packet circulating indefinitely: the identifier cache, the traversal path carried in the header, and the hop allowance. Any one of them alone would suffice; all three are applied because each covers a case the others do not.
- A packet with no available next hop is retained rather than dropped. This is the point at which forwarding hands over to Algorithm 8.

5.6 Requirements Satisfied

FWD-1, FWD-2, FWD-3, FWD-4, FWD-5, FWD-6, FWD-7, REL-1, REL-3, SEC-2, SEC-4, SEC-5, LOG-3, SNF-1.

6 Algorithm 5: Priority Scheduling with Per-Hop Pre-emption

6.1 Purpose

An emergency message must reach its destination ahead of routine traffic, and it must keep that advantage at every node along the way, not only at the node where it was created. A first-in-first-out queue cannot provide this, because an emergency message arriving at a busy relay would wait behind whatever was already queued. The outgoing queue is therefore a binary min-heap ordered on the pair (priority, creation time), which gives precedence to emergency traffic and, within a single priority level, delivers the oldest message first.

6.2 Inputs and Outputs

Symbol	Direction	Meaning
p.priority	input	SOS = 0, NORMAL = 1, STATUS = 2. A lower value denotes higher urgency.
p.created_at	input	Creation time recorded by the originating node.
Q	output	Binary min-heap of packets awaiting transmission.

The priority names used here (SOS, NORMAL, STATUS) match `packet.py`*'s* `Priority` *values directly, rather than the EMERGENCY/NORMAL/STATUS naming used loosely elsewhere — SOS and EMERGENCY refer to the same priority level.*

6.3 Pseudocode

Algorithm 5 — PriorityScheduling

```
// The ordering key is a triple. The sequence counter is included only to
// break exact ties, so that the ordering is total and the heap is stable.

procedure ENQUEUE(p)
    key <- ( p.priority, p.created_at, self.seq )
    self.seq <- self.seq + 1
    HEAP_PUSH(Q, key, p)

function DEQUEUE()
    if Q is empty then
        return NIL
    return HEAP_POP(Q)                    // lowest priority value first, then oldest

// ---------- the transmitting loop ----------
procedure DRAIN()
    while Q is not empty do
        p <- DEQUEUE()
        h <- ROUTING.next_hop(p.dst)
        if h = NIL then
            STOREFWD.retain(p)            // route lost while queued
            continue
        TRANSPORT.send(NEIGHBOUR[h].address, ENCODE(p))
        if p originated here then
            RELIABILITY.await_ack(p)      // Algorithm 7 takes over

// Pre-emption needs no separate mechanism. It follows from the ordering:
//     ( 0, t ) < ( 1, t' )   for every t and every t'
// so an SOS packet arriving at a node whose queue already holds
// NORMAL traffic is placed at the head of that queue. Because ENQUEUE is
// called at every hop and not only at the originator, the precedence is
// re-applied at each node the packet crosses (PRI-3).
```

6.4 Complexity

Both ENQUEUE and DEQUEUE are O(log n) in the number of queued packets, and DRAIN is O(n log n) to empty a queue of n packets. Space is O(n). Because pre-emption is a property of the ordering rather than a separate scan of the queue, no additional cost is incurred when an emergency message arrives. The current build proves this ordering directly with an automated test (`test_sos_forwarded_before_already_queued_normal`), which confirms an SOS queued after a NORMAL packet is still sent first.

6.5 Requirements Satisfied

PRI-1, PRI-2, PRI-3, PRI-4, PRI-5, PRI-6.

7 Algorithm 6: Duplicate Suppression and Loop Prevention

7.1 Purpose

In a mesh in which every node forwards for every other node, a single message can reach the same node by more than one path. Without suppression, each arrival would be forwarded again and the number of copies in flight would grow without bound. The algorithm keeps a time-bounded record of the message identifiers it has already observed and discards any repeat, while a periodic sweep keeps the record from growing indefinitely.

7.2 Inputs and Outputs

Symbol	Direction	Meaning
T_keep	input	Period for which an observed identifier is remembered.
SEEN	output	Map from message identifier to the time of first observation.

7.3 Pseudocode

Algorithm 6 — SuppressDuplicates

```
function IS_DUPLICATE(msg_id)
    return msg_id in SEEN

procedure RECORD(msg_id)
    SEEN[msg_id] <- NOW()

procedure SWEEP()                          // periodic; keeps the record bounded
    for each (id, t) in SEEN do
        if NOW() - t > T_keep then
            REMOVE id from SEEN

// ---------- why three guards and not one ----------
// 1. SEEN stops a copy that arrives at this node a second time, even by
//    a completely different path. It is the only guard that catches a
//    duplicate created by two disjoint routes.
// 2. p.path stops a loop forming within one journey, and does so without
//    any memory at the node -- the evidence travels inside the packet.
// 3. p.ttl bounds the journey absolutely. It is the guard of last resort
//    and holds even if a node has just restarted and lost its SEEN cache.
```

7.4 Complexity

Lookup and insertion are O(1) using a hash map. The sweep is O(|SEEN|) and runs on a fixed schedule; the implemented version sweeps lazily on each lookup rather than on a separate timer, which is equivalent in effect. The size of the record is bounded by the message rate multiplied by T_keep, so memory does not grow with the running time of the node, which matters because a node may be left running for the whole duration of an incident.

7.5 Requirements Satisfied

REL-1, REL-2, REL-3, FWD-3, FWD-5, LOG-3.

8 Algorithm 7: Acknowledgement and Bounded Retransmission

8.1 Purpose

A message that is transmitted is not necessarily delivered. The destination returns an acknowledgement along the reverse path; if none arrives within the timeout, the originator recomputes the route and transmits again. The number of attempts is bounded, so a message addressed to a node that has genuinely gone cannot generate traffic indefinitely. Recomputing the route before each retry is what makes the retry useful: retransmitting along a path that has already failed would usually fail again.

8.2 Inputs and Outputs

Symbol	Direction	Meaning
T_ack	input	Time to wait for an acknowledgement before retrying.
A_max	input	Maximum number of delivery attempts per message.
PENDING	output	Messages transmitted and awaiting acknowledgement, with their deadlines.
status	output	Every originated message finishes as DELIVERED or FAILED.

8.3 Pseudocode

Algorithm 7 — ReliableDelivery

```
procedure SEND(m)
    m.attempts <- 0
    TRANSMIT(m)

procedure TRANSMIT(m)
    m.attempts <- m.attempts + 1
    h <- ROUTING.next_hop(m.dst)
    if h = NIL then
        STOREFWD.retain(m)               // hand over to Algorithm 8
        return
    TRANSPORT.send(NEIGHBOUR[h].address, ENCODE(m))
    PENDING[m.msg_id] <- ( m, NOW() + T_ack )
    m.status <- IN_TRANSIT

procedure ON_ACK(a)
    if a.msg_id not in PENDING then
        return                           // late or duplicate ack
    ( m, _ ) <- PENDING[a.msg_id]
    REMOVE a.msg_id from PENDING
    m.status    <- DELIVERED
    m.hop_count <- LENGTH(a.path)
    STORE.save_message(m)
    UI.push(m)

procedure CHECK_TIMEOUTS()               // runs on a fixed polling interval
    for each ( id, ( m, deadline ) ) in PENDING do
        if NOW() < deadline then
            continue
        REMOVE id from PENDING
```

```
ROUTING.record_failure(m.last_hop)   // feeds F[] in Algorithm 3
if m.attempts >= A_max then
    m.status <- FAILED
    LOG(failed, m, "attempt limit reached")
    STORE.save_message(m)
else
    ROUTING.recompute()           // the old path may be gone
    TRANSMIT(m)                   // retry on a fresh route
```

8.4 Complexity

Acknowledgement handling is O(1). The timeout sweep is O(|PENDING|) per period. The total number of transmissions caused by one message is at most A_max, so retransmission is bounded by construction; this is the guarantee required by SRS 5.1, that the system continue to operate under packet loss without unbounded retransmission.

8.5 Design Notes

- A failed attempt is recorded against the link that was used, and that record enters the cost function of Algorithm 3 through the term beta × f(u, v). A link that repeatedly fails therefore becomes progressively more expensive and is avoided without being declared dead. The current build calls `ROUTING.record_failure` on every timeout and applies it to the graph immediately, rather than waiting for the next periodic gossip report (§1.7) — proven by a test where three consecutive failures on a link cause Dijkstra to route around it on the very next lookup.
- Acknowledgements are themselves ordinary packets and are subject to the same validation, priority ordering and hop allowance as any other traffic.

8.6 Requirements Satisfied

REL-4, REL-5, REL-6, REL-7, RTE-4, MSG-6, MSG-7, LOG-2.

9 Algorithm 8: Store-and-Forward Retention and Re-attempt

9.1 Purpose

In a disaster the network is partitioned and a route to the destination may simply not exist at the moment a message is sent. Discarding the message would be the ordinary behaviour of a network stack and would be wrong here: connectivity is intermittent, and a route that does not exist now may exist in a minute when a rescue team walks into range. The message is therefore held and re-attempted whenever the topology changes. The queue is bounded in both count and age so that retention cannot exhaust the device.

Implementation status: *the current build implements all of 9.3 as specified, including the count bound, priority-based eviction, and the separate age-based EXPIRE sweep (§1.7).*

9.2 Inputs and Outputs

Symbol	Direction	Meaning
C_max	input	Maximum number of messages held in the retention queue.

Symbol	Direction	Meaning
T_hold	input	Maximum period for which a message is held.
R	output	Retention queue of currently undeliverable messages.

9.3 Pseudocode

Algorithm 8 — StoreAndForward

```
procedure RETAIN(m)
    m.status     <- RETAINED
    m.retained_at <- NOW()
    R <- R + [ m ]
    STORE.save_message(m)                 // survives a node restart
    UI.push_retained_count(LENGTH(R))     // SNF-6

    if LENGTH(R) > C_max then             // bound by count
        // evict the LOWEST priority message, that is, the one with the
        // LARGEST priority value; ties broken by the oldest retained_at.
        v <- argmax over R of ( m.priority, -m.retained_at )
        REMOVE v from R
        v.status <- FAILED
        LOG(evicted, v, "retention queue full")

procedure REATTEMPT()   // on node_arrived, on route change, and periodically
    for each m in R, taken in ( priority, retained_at ) order do
        if ROUTING.next_hop(m.dst) != NIL then
            REMOVE m from R
            RELIABILITY.send(m)           // back to Algorithm 7

procedure EXPIRE()                        // bound by age
    for each m in R do
        if NOW() - m.retained_at > T_hold then
            REMOVE m from R
            m.status <- EXPIRED
            LOG(expired, m, "retention period elapsed")
            STORE.save_message(m)
```

9.4 Complexity

RETAIN is O(|R|) as written because of the eviction scan, and reduces to O(log |R|) if the queue is held as a max-heap on the eviction key. REATTEMPT and EXPIRE are each O(|R|). Since |R| is bounded above by C_max, all three are bounded in absolute terms regardless of how long the node has been running — the bound is enforced by the current build (§1.7).

9.5 Design Notes

- Eviction removes the *lowest* priority message first. An emergency message is therefore never displaced by routine traffic, which is the behaviour required by SNF-5 read together with the business rules of SRS 5.5.
- Re-attempt is triggered by the node_arrived event raised by Algorithm 1, so a retained message is sent within one beacon period of a route becoming available rather than waiting for the next periodic sweep (SNF-3).
- Retained messages are written to the local database, so a node that is restarted does not lose traffic it had accepted responsibility for.

9.6 Requirements Satisfied

SNF-1, SNF-2, SNF-3, SNF-4, SNF-5, SNF-6, LOG-1, LOG-3.

10 Algorithm 9: Message Protection and Verification

10.1 Purpose

Because every message is carried by strangers' devices, the contents must be unreadable to the nodes that relay them, and an emergency message must not be forgeable. The payload is encrypted with an authenticated symmetric cipher, and the header together with the traversal path is signed. A relay can read only the header fields it needs in order to forward; it cannot recover the plaintext and cannot alter the packet without the alteration being detected at the destination.

Implementation status: *both halves are implemented and tested end to end (§1.7). AES-256-GCM payload encryption uses a pre-shared session key rather than the per-message X25519-derived key shown in 10.3. Ed25519 signing and verification are wired into the live send/receive path, with one deliberate deviation from 10.3's pseudocode: the signature covers only the fields that stay constant across every hop (not the literal "header + path"), since ttl, hop_count and path are each mutated by every relay by design — a signature over them could never verify past the first hop. Public keys are distributed via HELLO and propagate network-wide through Algorithm 2's re-flooding, so a destination several hops from the originator can still verify its signature.*

10.2 Inputs and Outputs

Symbol	Direction	Meaning
K_dst	input	Public key of the destination, learned from its discovery beacon.
k_sign	input	This node's Ed25519 private signing key, never transmitted.
p.payload	output	Nonce, ciphertext and authentication tag.
p.sig	output	Ed25519 signature over the header and the traversal path.

10.3 Pseudocode

Algorithm 9 — ProtectMessage and UnprotectMessage

```
function PROTECT(m, K_dst)
    k_shared <- X25519(self.private_key, K_dst)   // key agreement
    k_enc    <- HKDF(k_shared, info = m.msg_id)    // per-message key
    nonce    <- RANDOM(12 bytes)                   // never reused
    (ct, tag) <- AES_256_GCM_ENCRYPT(k_enc, nonce, m.plaintext)
    p.payload <- nonce || ct || tag
    p.header  <- BUILD_HEADER(m)                   // fixed 52-byte layout
    p.sig     <- Ed25519_SIGN(k_sign, p.header || p.path)
    return p

function UNPROTECT(p, K_src)
    if not Ed25519_VERIFY(K_src, p.header || p.path, p.sig) then
        return REJECT                              // forged or header tampered
    k_shared <- X25519(self.private_key, K_src)
    k_enc    <- HKDF(k_shared, info = p.msg_id)
    (nonce, ct, tag) <- SPLIT(p.payload)
```

```
    plaintext <- AES_256_GCM_DECRYPT(k_enc, nonce, ct, tag)
    if the authentication tag did not verify then
        return REJECT                              // payload modified in transit
    return plaintext

// ---------- what a relay does ----------
// A relaying node calls NEITHER procedure. It reads only the header fields
// it needs in order to forward -- destination, priority, hop allowance and
// traversal path -- and passes the payload through untouched. It holds no
// key with which the payload could be decrypted (SEC-2, FWD-7).
//
// All primitives are taken from an established cryptographic library. No
// primitive is implemented from first principles (SEC-7).
```

10.4 Complexity

Encryption and decryption are O(n) in the length of the payload. Key agreement, signing and verification are constant in the payload size and are performed once per message at the endpoints only. A relay performs none of these operations, so the cryptographic cost of the system does not grow with the number of hops a message crosses.

10.5 Requirements Satisfied

SEC-1, SEC-2, SEC-3, SEC-4, SEC-5, SEC-6, SEC-7, FWD-7.

11 Algorithm 10: Terminal Selection for Rescue-Addressed Traffic

11.1 Purpose

A user raising an SOS does not know, and should not have to know, which node will receive it. The message is addressed to the rescue service rather than to a named node, and the routing layer resolves that abstract destination to a concrete terminal. Both rescue nodes and gateway nodes are valid terminals; where more than one is reachable, the least-cost one is selected using the same weighted metric as ordinary routing.

11.2 Inputs and Outputs

Symbol	Direction	Meaning
dist[]	input	Least-cost distances produced by Algorithm 3.
G	input	Network graph carrying the advertised role of each node.
t	output	The chosen terminal, or NIL if no exit is currently reachable.

11.3 Pseudocode

Algorithm 10 — SelectTerminal

```
function SELECT_TERMINAL()
    T <- { v in G : role(v) = RESCUE or role(v) = GATEWAY }
```

```
    if T is empty then
        return NIL                          // no exit exists in the mesh
    t <- the v in T minimising dist[v]
    if dist[t] = INFINITY then
        return NIL                          // none currently reachable
    return t

procedure ON_SOS(m)
    m.priority <- SOS
    m.dst      <- RESCUE_SERVICE            // abstract destination
    t <- SELECT_TERMINAL()
    if t = NIL then
        STOREFWD.retain(m)                  // hold until a rescue arrives
        UI.show(m, QUEUED)
        return
    m.resolved_dst <- t
    RELIABILITY.send(m)

// A rescue node that walks into the affected area is discovered by
// Algorithm 1 within one beacon period, raises node_arrived, and the
// retained SOS messages are re-attempted immediately by Algorithm 8.
// No configuration is required on either side (SRS 2.3).
```

11.4 Complexity

O(|T|) after route computation has completed, where |T| is the number of nodes advertising the gateway or rescue role — in practice a small constant. The selection adds nothing measurable to the cost of sending a message. In the implemented version this selection is folded directly into Dijkstra (the cheapest node with a matching role is chosen during the same traversal) rather than run as a separate post-processing step; the complexity is unaffected. As noted in §1.7, a terminal is only found if it lies within the graph this node currently knows about.

11.5 Requirements Satisfied

RTE-6, RTE-7, MSG-2, MSG-3, PRI-5, SNF-3, DSH-3.

12 Complexity Summary

The table below collects the time and space behaviour of every algorithm. V is the number of nodes in the network graph, E the number of links, |N| the number of direct neighbours, and n the number of items held in the structure concerned.

No.	Algorithm	Time	Space	Bounded By
1	Neighbour Discovery	O(|N|) per period	O(|N| × W)	Neighbour count
2	Link-State Dissemination	O(|N| + deg) per report	O(V + E)	Sequence numbers
3	Route Computation	O((V + E) log V)	O(V + E)	Graph size
4	Packet Forwarding	O(TTL_0) per packet	O(1)	Hop allowance
5	Priority Scheduling	O(log n) per operation	O(n)	Queue length

No.	Algorithm	Time	Space	Bounded By
6	Duplicate Suppression	O(1) lookup, O(n) sweep	O(n)	T_keep
7	Reliable Delivery	O(1) per ack	O(n)	A_max
8	Store-and-Forward	O(|R|), O(log |R|) with a heap	O(C_max)	C_max and T_hold
9	Message Protection	O(payload length)	O(payload length)	Payload size
10	Terminal Selection	O(|T|)	O(|T|)	Terminal count

Every structure in the system is bounded by a configured parameter rather than by the running time of the node, and every bound in this table is enforced by the current build (§1.7). This is a deliberate property: a node may be left running for the whole duration of an incident on battery power, and no data structure is permitted to grow without limit while it does so.

13 Traceability to the SRS

The matrix below records which algorithm realises each functional requirement group of Section 4 of the Software Requirements Specification. Every group is covered.

SRS Feature	Requirement IDs	Realised By
4.1 Peer Discovery	DIS-1 – DIS-7	Algorithm 1
4.2 Message Composition and Transmission	MSG-1 – MSG-7	Algorithms 7, 10
4.3 Multi-Hop Message Forwarding	FWD-1 – FWD-7	Algorithms 4, 6, 9
4.4 Adaptive Routing	RTE-1 – RTE-8	Algorithms 2, 3, 10
4.5 Priority-Based Message Handling	PRI-1 – PRI-6	Algorithms 5, 10
4.6 Reliability Mechanisms	REL-1 – REL-8	Algorithms 1, 4, 6, 7
4.7 Store-and-Forward Delivery	SNF-1 – SNF-6	Algorithms 4, 8, 10
4.8 Message Security	SEC-1 – SEC-7	Algorithms 1, 4, 9
4.9 Message Logging and History	LOG-1 – LOG-6	Algorithms 4, 6, 7, 8
4.10 Rescue Monitoring Dashboard	DSH-1 – DSH-6	Algorithms 5, 10
5.1 Performance Requirements	—	Algorithms 2, 3, 5
5.2 Safety Requirements	—	Algorithms 4, 6, 8
5.3 Security Requirements	—	Algorithms 4, 9
5.5 Business Rules	—	Algorithms 4, 5, 8, 10